



13

Building an Event-Based Backend Using Express and Socket.IO

In this chapter, we will learn about event-based applications and the tradeoffs of using such an architecture versus a more traditional one. Then, we are going to learn about WebSockets and how they work. Afterward, we are going to implement a backend using Socket.IO and Express. Finally, we are going to learn how to integrate authentication by using JWT with Socket.IO.

In this chapter, we are going to cover the following main topics:

- What are event-based applications?
- Setting up Socket.IO
- Creating a backend for a chat app using Socket.IO
- Adding authentication by integrating JWT with Socket.IO

Technical requirements

Before we start, please install all the requirements from [Chapter 1, Preparing For Full-Stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in this book. While installing a newer version should not be an issue, please note that certain steps might work differently. If you are having an issue

with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1* and *2*.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch13>.

If you cloned the full repository for this book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

The CiA video for this chapter can be found at:

<https://youtu.be/kHGvkoopIHf4>.

What are event-based applications?

In contrast to traditional web applications, where we have a request-response pattern, in event-based applications, we are dealing with events. The server and client stay connected and each side can send events, which the other side listens to and reacts to.

The following diagram shows the difference between implementing a chat app in a request-response pattern versus an event-based pattern:

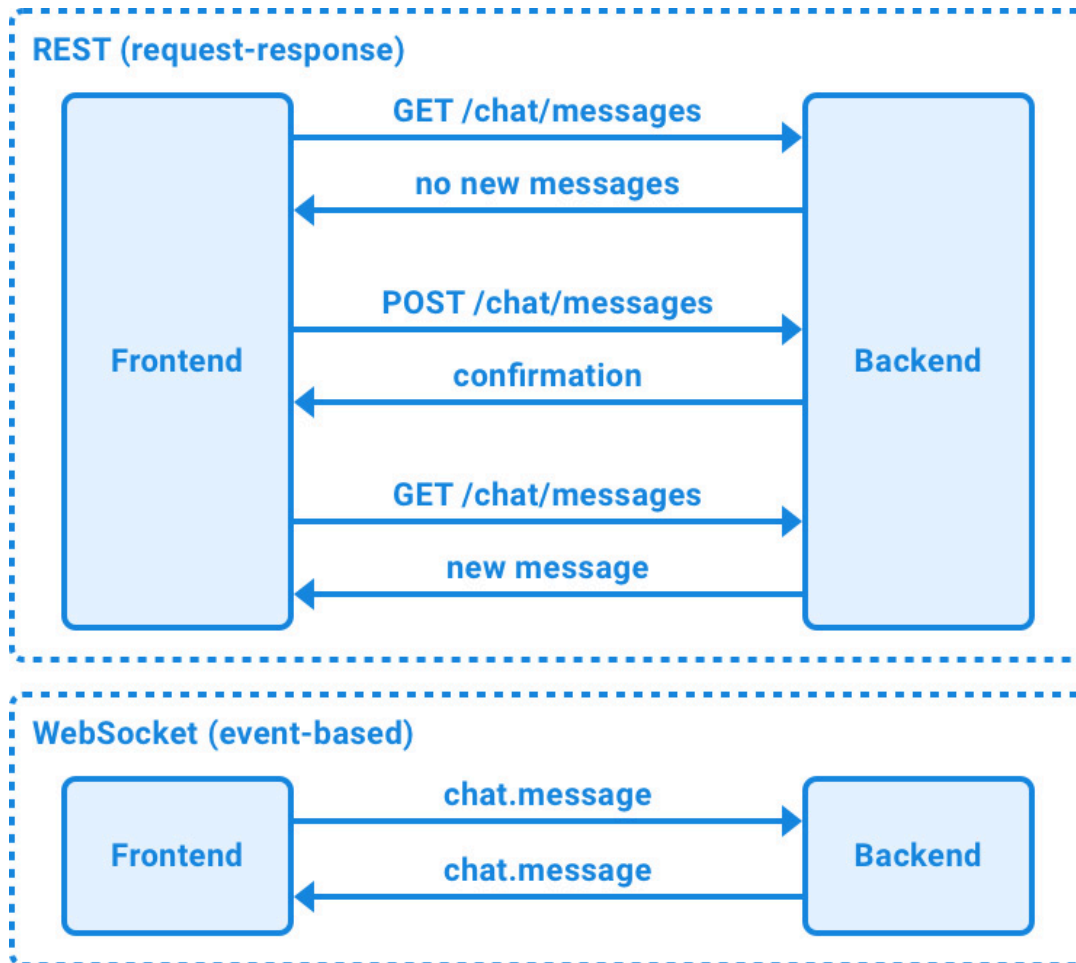


Figure 13.1 – A chat app implementation with request-response and event-based patterns

For example, to implement a chat application in a request-response pattern, we would need to regularly send a request to a `GET /chat/messages` endpoint to refresh the list of messages sent in a chat room. This process of periodically sending requests is called short polling. To send a chat message, we would make a request to `POST /chat/messages`. In an event-based pattern, we could instead send a `chat.message` event from the client to the server, which then sends a `chat.message` event to all connected users. The clients then listen to `chat.message` events and display the messages as they come in; no periodic requests are required!

Of course, each pattern comes with its advantages and disadvantages:

- REST/request-response:
 - Good for when data does not change frequently

- Responses can be cached easily
- Requests are stateless, making it easy to scale backends
- Bad at real-time updates (requires periodic polling)
- More overhead per request (bad when sending many short responses)
- WebSockets/event-based:
 - Good for applications that require frequent updates
 - More efficient because a persistent connection between the client and server is re-used for multiple requests
 - Less overhead per request
 - There might be connection issues with (corporate) proxies
 - They are stateful, which can make it harder to scale an app

As we can see, for fetching data that does not change so frequently (and can be cached), such as blog posts, a request-response pattern fits better. For applications where data is frequently changing, such as a chat room, an event-based pattern fits better.

What are WebSockets?

The WebSocket API is a browser feature that allows web applications to create an open connection between the client and the server, similar to Unix-style sockets. With WebSockets, communication can happen in both directions at the same time. This is in contrast to HTTP requests, where both parties can communicate, but not simultaneously.

WebSockets use HTTP to establish a connection between the client and the server, and then upgrade the protocol from HTTP to the WebSocket protocol. While both HTTP and WebSockets depend on the **Transmission Control Protocol (TCP)**, they are distinct protocols on the application layer (Layer 7) of the **Open Systems Interconnection (OSI)** model.

A connection to a WebSocket is established by sending an HTTP request with the **Upgrade: websocket** header and other parameters to establish a secure WebSocket connection. The server then responds with an **HTTP**

101 Switching Protocols response code and information to establish the connection. Then, the client and server continue talking on the WebSocket protocol.

What is Socket.IO?

Socket.IO is an implementation of an event-based server and client library. In most cases, it establishes a connection to the server using a WebSocket. If a WebSocket connection is not possible (due to lacking browser support or firewall settings), Socket.IO can also fall back to HTTP long-polling. However, Socket.IO isn't a pure WebSocket implementation as it adds additional metadata to each packet. It only uses WebSockets internally to transmit data.

In addition to providing a way to send events between the client and server, Socket.IO offers the following features over plain WebSockets:

- **Fallback to HTTP long-polling:** This occurs if the WebSocket connection cannot be established. This can be especially useful for companies using proxies or firewalls that block WebSocket connections.
- **Automatic reconnection:** If the WebSocket connection is interrupted.
- **Buffering packets:** When the client gets disconnected, packets can be re-sent again automatically upon reconnection.
- **Acknowledgments:** A convenient way to send events in a request-response pattern, which can sometimes be useful even in event-based applications.
- **Broadcasting:** Sending an event to all (or a subset of all) connected clients.
- **Multiplexing:** Socket.IO implements namespaces, which can be used to create “channels” that only certain users can send events to and receive events from, such as an “admin-only channel.”

Now that we have learned the essentials of what Socket.IO is, let's dive deeper into how the connection and emitting/receiving events work.

Connecting to Socket.IO

The following diagram shows how a connection is established with Socket.IO:

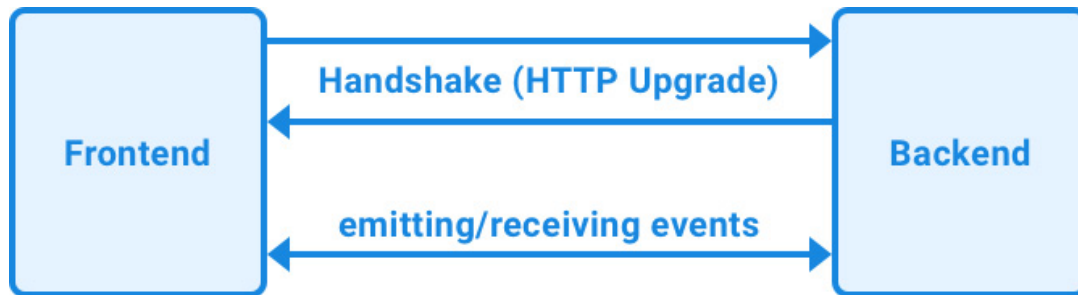


Figure 13.2 – Establishing a connection with Socket.IO

First, Socket.IO sends a handshake from the client (on the frontend) to the server (on the backend), which can contain information to authenticate with the server, or query parameters to provide additional information when establishing the connection.

If a connection via WebSockets cannot be established, Socket.IO connects to the server via HTTP long polling, which means making a request to the server that is kept active until an event occurs, at which point the server sends a response to the request. This allows waiting for events without having to periodically make a request to see if there are any new events. Of course, this is not as performant as WebSockets, but it is a good fallback for when WebSockets aren't available.

Emitting and receiving events

Once connected to Socket.IO, we can start **emitting** (sending) and receiving events. Events are handled by registering event handler functions, which are called when a certain type of event is received by either the client or the server. Both the client and the server can emit and receive events. Additionally, events can be **broadcast** from the server to multiple clients. The following diagram shows an example of how events are emitted and received in a chat application:



Figure 13.3 – Emitting and receiving events with Socket.IO

As we can see, **User 1** sends a **Hello everyone** message, which the server (backend) then broadcasts to all other clients (frontend). In this case, the message gets broadcast back to **User 1**, as well as to **User 2**.

If we want to restrict the clients that receive certain events, Socket.IO allows the creation of **rooms**. Clients can join a room, and on the server, we can also broadcast events to only specific rooms. This concept can be used for chat rooms, but also for collaborating on a specific project (such as editing a document together in real time).

In addition to emitting and receiving events asynchronously, Socket.IO offers a way to send an event that expects a response via **acknowledgments**. These can be used to model request-response patterns in Socket.IO. For example, we could request information about a certain user using a **user.info** event and synchronously wait for the server response (acknowledgment). We can see this in the preceding diagram, where **User 2** requests information about a certain user, and then gets a response containing the user's information.

Now that we've learned about event-based applications, WebSockets, and Socket.IO, let's put this theory into practice and set up Socket.IO.

Setting up Socket.IO

To set up the Socket.IO server, we are going to base our code on what we had in [Chapter 6, Adding Authentication and Roles with JWT](#), as it already includes some boilerplate for a backend and frontend with JWT authentication. Later in this chapter, in the *Adding authentication by integrating JWT with Socket.IO* section, we are going to make use of JWT to add authentication to Socket.IO:

1. Copy the existing **ch6** folder to a new **ch13** folder, as follows:

```
$ cp -R ch6 ch13
```

2. Open the **ch13** folder in VS Code.
3. Now, we can start setting up Socket.IO. First, install the **socket.io** package in the backend folder by running the following commands:

```
$ cd backend/  
$ npm install socket.io@4.7.2
```

4. Edit **backend/.env** and change **DATABASE_URL** so that it points to a new **chat** database:

```
DATABASE_URL=mongodb://localhost:27017/chat
```

5. Edit **backend/src/app.js** and import the **createServer** function from **node:http** and the **Server** function from **socket.io**:

```
import { createServer } from 'node:http'  
import { Server } from 'socket.io'
```

We are going to need to create a **node:http** server as we cannot directly connect Socket.IO to Express. Instead, Socket.IO attaches to a **node:http** server.

6. Thankfully, Express can also be easily attached to a **node:http** server. Edit **backend/src/app.js** and, before **app** is exported, create a new **node:http** server from the Express app, as follows:


```
const server = createServer(app)
```

7. Now, create a new Socket.IO server from the **node:http** server:

```
const io = new Server(server, {  
  cors: {  
    origin: '*',  
  },  
})
```

WARNING

*Setting the origin to * makes it possible for phishing sites to imitate your website and send requests to your backend. In production, the origin should be set to the deployed URL of your frontend.*

8. We can use the Socket.IO server to listen to connections from clients and print a message:

```
io.on('connection', (socket) => {  
  console.log('user connected:', socket.id)
```

9. The active client connection can be tracked by using the **socket** object. For example, we can listen to disconnect events from the client like so:

```
socket.on('disconnect', () => {  
  console.log('user disconnected:', socket.id)  
})  
})
```

10. Lastly, change the export so that it uses the **node:http** server instead of the Express app directly:

```
export { server as app }
```

11. Start the backend by running the following commands:

```
$ cd backend/  
$ npm run dev
```

Don't forget to get Docker and the database container up and running before starting the backend. Keep the backend running for the rest of this chapter.

Now that we have set up a simple Socket.IO server, let's continue by setting up the client.

Setting up a simple Socket.IO client

We are going to use the existing frontend for now. In the next chapter, [*Chapter 14, Creating a Frontend to Consume and Send Events*](#), we are going to remove the blog components and create a new React frontend for our chat app. Let's start setting up a simple Socket.IO client:

1. In the root of the project, install the **socket.io-client** package for the frontend by running the following command:

```
$ npm install socket.io-client@4.7.2
```

Make sure that you aren't in the **backend** folder anymore!

2. Edit **src/App.jsx** and import the **io** function from **socket.io-client**:

```
import { io } from 'socket.io-client'
```

3. Define a new instance of the Socket.IO client by using the **io** function and passing a hostname and port:

```
const socket = io(import.meta.env.VITE_SOCKET_HOST)
```

Here, we will be passing **localhost:3001** through an environment variable. We cannot pass the HTTP URL here as Socket.IO will try to connect to the hostname and port using WebSockets.

4. Listen to the **connect** event and print out a message if we successfully connected to the Socket.IO server:

```
socket.on('connect', () => {  
  console.log('connected to socket.io as', socket.id)  
})
```

5. Additionally, listen to the **connect_error** event and log an error message in case connecting to the Socket.IO server failed:

```
socket.on('connect_error', (err) => {  
  console.error('socket.io connect error:', err)  
})
```

6. Edit **.env** and add the following environment variable:

```
VITE_SOCKET_HOST="localhost:3001"
```

7. Run the frontend, as follows:

```
$ npm run dev
```

8. Now, open the frontend in your browser by going to **http://localhost:5173/**. Keep the frontend running for the rest of this chapter.

You will see a message stating **connected to socket.io** in the browser console. In the server output, you will see that the client connected successfully. Try refreshing the page to see it disconnecting and connecting again (with a new socket ID):

```
successfully connected to database: mongodb://localhost:27017/ch3  
express server running on http://localhost:3001  
user connected: 8c0lseEPbgQ8d4AzAAAB  
user disconnected: 8c0lseEPbgQ8d4AzAAAB  
user connected: rmGIXt3HV8S1qusRAAAD
```

Figure 13.4 – Seeing the Socket.IO client connect to and disconnect from our server

Now that we have successfully set up a Socket.IO server, let's continue by creating a backend for a chat app using Socket.IO.

Creating a backend for a chat app using Socket.IO

We can now start implementing a chat app using Socket.IO. We will be developing the following functionality for our chat app:

- **Emitting** events to send chat messages from the client to the server
- **Broadcasting** chat messages from the server to all clients
- Joining **rooms** to send messages in
- Using **acknowledgments** to get information about a user

Let's get started!

Emitting events to send chat messages from the client to the server

We'll start by **emitting** a `chat.message` event from the client to the server. For now, we are going to emit this event right after connecting. Later, we are going to integrate this into a frontend. Follow these steps to send chat messages from the client and receive them on the server:

1. Edit **backend/src/app.js** and *cut/remove* the following code:

```
io.on('connection', (socket) => {  
  console.log('user connected:', socket.id)  
  socket.on('disconnect', () => {  
    console.log('user disconnected:', socket.id)  
  })  
})
```

2. Create a new **backend/src/socket.js** file, define a **handleSocket** function there, and paste the following code inside it:

```
export function handleSocket(io) {
  io.on('connection', (socket) => {
    console.log('user connected:', socket.id)
    socket.on('disconnect', () => {
      console.log('user disconnected:', socket.id)
    })
  })
}
```

3. Now, add a new listener that listens to the **chat.message** event and logs the message sent from the client:

```
socket.on('chat.message', (message) => {
  console.log(`${socket.id}: ${message}`)
})
}
```

4. Edit **backend/src/app.js** and import the **handleSocket** function:

```
import { handleSocket } from './socket.js'
```

5. Once the Socket.IO server has been created, call the **handleSocket** function:

```
const io = new Server(server, {
  cors: {
    origin: '*',
  },
})
handleSocket(io)
```

6. Edit **src/App.jsx** and emit a **chat.message** event with some text, as follows:

```
socket.on('connect', () => {
  console.log('connected to socket.io as', socket.id)
  socket.emit('chat.message', 'hello from client')
})
```

INFO

Socket.IO allows us to send any kind of serializable data structures in an event, not just strings! For example, it is possible to send objects and arrays.

The backend and frontend should automatically refresh and the server will log the following message:

```
XXmWHjA_5zew70VIAAAM: hello from client
```

If not, make sure you (re-)start the backend and frontend and refresh the page manually.

As you can see, it is quite simple to send and receive events asynchronously in real time using Socket.IO.

Broadcasting chat messages from the server to all clients

Now that the backend server can receive messages from a client, we need to **broadcast** the messages to all other clients so that others can see the chat messages that were sent. Let's do that now:

1. Edit **backend/src/socket.js** and extend the **chat.message** event listener so that it calls **io.emit** and sends the chat message to everyone:

```
socket.on('chat.message', (message) => {  
  console.log(`${socket.id}: ${message}`)  
  io.emit('chat.message', {  
    username: socket.id,  
    message,  
  })  
})
```

```
    })  
  })
```

NOTE

Alternatively, you can use `socket.broadcast.emit` to send an event to every client except the one that sent the message.

2. We also need to add a listener for chat messages on the client side. This works the same way as on the server. Edit `src/App.jsx` and add the following event listener:

```
socket.on('chat.message', (msg) => {  
  console.log(`${msg.username}: ${msg.message}`)  
})
```

3. Now, you should see the message being logged on the server and the client. Try opening a second window; you will see messages from both clients in your browser!

connected to socket.io as ewy3yhSnI_59fGhbAAAj	<u>App.jsx:13</u>
ewy3yhSnI_59fGhbAAAj: hello from client	<u>App.jsx:17</u>
mYE_P9IrsqF0mVzaAAAL: hello from client	<u>App.jsx:17</u>

Figure 13.5 – Receiving messages from another client

Joining rooms to send messages in

While having a working chat where messages get relayed to everyone is nice, often, we don't want to broadcast our messages to everyone. Instead, we might want to only send messages to a certain group of people. To facilitate this, Socket.IO provides **rooms**. Rooms can be used to group clients together so that events are only sent to all other clients in the room. This feature can be used to create chat rooms, but also for collaborating on a project together (by creating a new room for each project). Let's learn how rooms can be used in Socket.IO:

1. Socket.IO allows us to pass a query string during the handshake. We can access this query string to get the room that the client wants to join. Edit **backend/src/socket.js** and get the room from the handshake query:

```
io.on('connection', (socket) => {  
  console.log('user connected:', socket.id)  
  const room = socket.handshake.query?.room ?? 'public'
```

2. Now, use **socket.join** to join the client into the selected room:

```
socket.join(room)  
console.log(socket.id, 'joined room:', room)
```

3. Then, inside the **chat.message** handler, use **.to(room)** to make sure chat messages from that client are only sent to a certain room:

```
io.to(room).emit('chat.message', {  
  username: socket.id,  
  message,  
})
```

4. In the client, we need to pass a query string to tell the server which room we would like to join. Edit **src/App.jsx**, as follows:

```
const socket = io(import.meta.env.VITE_SOCKET_HOST, {  
  query: window.location.search.substring(1),  
})
```

The Socket.IO query string is a URL query string, so we can simply pass the query string of the current page to it (without **?** at the beginning of the string).

5. Open **http://localhost:5173/** and **http://localhost:5173/?room=test** in two separate browser windows and send messages from both. You will see that the message from the second window doesn't get sent to the first window. However, if you open another window with the **?room=test** query string and send a message there,

you will see the message being forwarded to the second window (but not the first).

As we can see, we can use rooms to have more fine-grained control over which clients receive certain events. As the server controls which rooms a client joins, we can also add permission checks before allowing a client to join a room.

Using acknowledgments to get information about a user

As we have seen, events are a great way to send asynchronous messages. Sometimes, however, we want a more traditional synchronous request-response API, like we had with REST. In Socket.IO, we can implement synchronous events by using **acknowledgments**. We can use acknowledgments to, for example, get more information about a user in the current chat room. For now, we are only going to return the rooms that the user is in. Later, when we add authentication, we are going to fetch the user object from the database here. Let's get started implementing acknowledgments:

1. Edit **backend/src/socket.js** and define a new event listener:

```
socket.on('user.info', async (socketId, callback) => {
```

Note how we are passing a callback function as the last argument. This is what makes the event an acknowledgment.

2. In this event listener, we are going to fetch all sockets in the room with the ID of our socket:

```
const sockets = await io.in(socketId).fetchSockets()
```

Internally, Socket.IO creates a room for each connected socket, to make it possible to send events to a single socket.

NOTE

We could directly access the sockets of the current instance, but that wouldn't work anymore when we scale our service to multiple instances in a cluster. To make it work even in a cluster, we need to use the room functionality to get a socket by ID.

3. Now, we must check if we found a socket with the given ID. If not, we return **null**:

```
if (sockets.length === 0) return callback(null)
```

4. Otherwise, we return the socket ID and a list of rooms that the user is in:

```
const socket = sockets[0]
const userInfo = {
  socketId,
  rooms: Array.from(socket.rooms),
}
return callback(userInfo)
})
```

5. Now, we can emit the **user.info** event on the client. Edit **src/App.jsx** and start by making the **connect** event listener an **async** function:

```
socket.on('connect', async () => {
  console.log('connected to socket.io as', socket.id)
  socket.emit('chat.message', 'hello from client')
```

6. To emit an event with an acknowledgment, we can use the **emitWithAck** function, which returns a Promise that we can **await**:

```
const userInfo = await socket.emitWithAck('user.info', socket.id)
console.log('user info', userInfo)
})
```

7. After saving the code, go to the browser window; you will see the user's information being logged in the console:

```
connected to socket.io as 3i1MTX28X7H8xtXYAAAB
3i1MTX28X7H8xtXYAAAB: hello from client
user info
▼ {socketId: '3i1MTX28X7H8xtXYAAAB', rooms: Array(2)} ⓘ
  ► rooms: (2) ['3i1MTX28X7H8xtXYAAAB', 'public']
    socketId: "3i1MTX28X7H8xtXYAAAB"
```

Figure 13.6 – Getting user information with an acknowledgment

Now that we have learned how to send various kinds of events, let's get into a more advanced topic: authentication with Socket.IO.

Adding authentication by integrating JWT with Socket.IO

So far, all chat messages have been sent with the socket ID as the “user-name.” This is not a very good way to identify users in a chat room. To fix this, we are going to introduce user accounts by authenticating sockets with JWT. Follow these steps to implement JWT with Socket.IO:

1. Edit **backend/src/socket.js** and import **jwt** from the **jsonwebtoken** package and **getUserInfoById** from our service functions:

```
import jwt from 'jsonwebtoken'
import { getUserInfoById } from './services/users.js'
```

2. Inside the **handleSocket** function, define a new Socket.IO middleware by using **io.use()**. Middleware in Socket.IO works similarly to middleware in Express – we define a function that runs before requests are processed, as follows:

```
export function handleSocket(io) {
  io.use((socket, next) => {
```

3. Inside this function, we check if the token was sent via the **auth** object (similar to how we passed **room** earlier via the query string). If no token was passed, we pass an error to the **next()** function and cause the connection to fail:

```
if (!socket.handshake.auth?.token) {  
  return next(new Error('Authentication failed: no token provided'))  
}
```

NOTE

*It is important not to pass a JWT via the query string since this is part of the URL. It is exposed in the browser address bar and thus potentially stored in the browser history, where it could be extracted by a potential attacker. Instead, the **auth** object is sent via the request payload during the handshake, which is not exposed in the address bar.*

4. Otherwise, we call **jwt.verify** to verify the token by using the existing **JWT_SECRET** environment variable:

```
jwt.verify(  
  socket.handshake.auth.token,  
  process.env.JWT_SECRET,
```

5. If the token is invalid, we once again return an error in the **next()** function:

```
async (err, decodedToken) => {  
  if (err) {  
    return next(new Error('Authentication failed: invalid token'))  
  }  
}
```

6. Otherwise, we save the decoded token to **socket.auth**:

```
socket.auth = decodedToken
```

7. Additionally, we fetch the user information from the database and, for convenience, store it in **socket.user**:

```
socket.user = await getUserInfoById(socket.auth.sub)  
return next()
```

```
    },  
  )  
})
```

NOTE

Make sure that `next()` is always called in `Socket.IO` middleware.

Otherwise, `Socket.IO` will keep the connection open until it is closed after a given timeout.

8. The `user` object contains a `username` value. Now, we can *replace* the socket ID in the chat message with the username:

```
    socket.on('chat.message', (message) => {  
      console.log(`${socket.id}: ${message}`)  
      io.to(room).emit('chat.message', {  
        username: socket.user.username,  
        message,  
      })  
    })  
  })
```

9. We can also return the user information from the `user.info` event:

```
const userInfo = {  
  socketId,  
  rooms: Array.from(socket.rooms),  
  user: socket.user,  
}
```

10. We still need to send the auth object from the client side, edit `src/App.jsx`, and get the token from `localStorage`, as follows:

```
const socket = io(import.meta.env.VITE_SOCKET_HOST, {  
  query: window.location.search.substring(1),  
  auth: {  
    token: window.localStorage.getItem('token'),  
  },  
})
```


(replacing `<JWT>` with the copied token). Upon refreshing the page, it should work! As we can see, when logged in with two different users, we can see their messages with their respective usernames:

```
connected to socket.io as BVas04Ss1_UsnVY3AAAN      App.jsx:17
test: hello from client                             App.jsx:28
user info                                           App.jsx:28
  ▶ {socketId: 'BVas04Ss1_UsnVY3AAAN', rooms: Array(2), user: {...}}
daniel: hello from client                          App.jsx:28
```

Figure 13.9 – Receiving messages from different users

Our chat backend is now fully functional! In the next chapter, we are going to create a frontend to complete our chat app.

Summary

In this chapter, we learned about event-based applications, WebSockets, and Socket.IO. Then, we set up Socket.IO on the backend (server) and frontend (client). Afterward, we learned how to send messages between the server and client, how to join rooms, and how to broadcast messages. We also used acknowledgments to get information about a user in a request-response pattern with Socket.IO. Finally, we implemented authentication using JWT in Socket.IO, finalizing our chat app backend.

In the next chapter, [***Chapter 14**, Creating a Frontend to Consume and Send Events*](#), we are going to create a frontend for our chat app, which is going to interact with the backend we created in this chapter.